

Regular Expressions in Php

Laxmikant Soni

www.medicaps.ac.in

Learning Objectives

- By the end of this lecture, students will:
 - Understand the concept and importance of regular expressions in PHP.
 - Use PHP functions such as `preg_match()`, `preg_match_all()`, `preg_replace()`, and `preg_split()` effectively.
 - Apply regular expressions to validate user input (e.g., emails, phone numbers, passwords).
 - Write and test patterns for searching, replacing, and extracting text from strings.



What is a Regular Expression?

A **regular expression** is a sequence of characters that forms a search pattern. When you search for data in a text, you can use this search pattern to describe what you are searching for.

A regular expression can be a single character, or a more complicated pattern.

Regular expressions can be used to perform all types of **text search** and **text replace** operations.



Syntax

In PHP, regular expressions are strings composed of **delimiters**, a **pattern**, and optional **modifiers**.

```
$exp = "/medicaps university/i";
```



Syntax

In the example above:

- `/` is the **delimiter**
- `medicaps university` is the **pattern** being searched for
- `i` is a **modifier** that makes the search case-insensitive

The delimiter can be any character that is not a letter, number, backslash, or space.

The most common delimiter is the forward slash (`/`), but when your pattern contains forward slashes it is convenient to choose other delimiters such as `#` or `~`.



Common Regular Expression Modifiers in PHP

Modifier	Description
i	Makes the pattern case-insensitive.
m	Treats the string as multi-line. ^ and \$ match the start and end of each line.
s	Enables “dotall” mode, where . matches newline characters as well.
u	Enables Unicode support (UTF-8).
x	Allows whitespace and comments inside the pattern for readability.
A	Forces the match to occur only at the beginning of the string.
D	Forces \$ to match only at the end of the string.
U	Inverts the “greedy” behavior of quantifiers, making them lazy by default.



Regular Expression Functions

Function	Description
<code>preg_match()</code>	Returns 1 if the pattern was found in the string and 0 if not.
<code>preg_match_all()</code>	Returns the number of times the pattern was found in the string (may be 0).
<code>preg_replace()</code>	Returns a new string where matched patterns have been replaced with another string.
<code>preg_split()</code>	Splits a string into an array using a regular expression as the delimiter.
<code>preg_grep()</code>	Returns an array of elements from the input array that match a given pattern.
<code>preg_filter()</code>	Returns an array with matched elements replaced, similar to <code>preg_replace()</code> but only includes matches.



Example: preg_match ()

The `preg_match ()` function searches a string for a pattern and returns **1 if a match is found**, and **0 if not**.

```
<?php
$pattern = "/php/i";    // 'i' makes the search
                        case-insensitive
$text = "I love learning PHP!";

if (preg_match($pattern, $text)) {
    echo "Match found!";
} else {
    echo "No match found!";
}
?>
```



Example: preg_match_all()

The `preg_match_all()` function searches a string for a pattern and returns the **number of matches found**. It can also return the matches themselves if you pass an array variable.

```
<?php
$pattern = "/\d+"/;    // pattern to match one or more
                        digits
$text = "There are 2 apples, 5 bananas, and 12
oranges.";

if (preg_match_all($pattern, $text, $matches)) {
    echo "Total matches found: " . count($matches[0]) .
"\n";
    print_r($matches[0]); // print all matched numbers
} else {
    echo "No matches found!";
}
```



Explanation:

- The pattern `//` matches one or more digits.
- `preg_match_all()` finds all matches in the string, not just the first one.
- The `$matches[0]` array contains all the matched values.



Example: preg_replace()

The `preg_replace()` function searches a string for a pattern and **replaces** all matches with a new string.

```
<?php
```

```
$pattern = "/apple/i";    // 'i' makes it case-  
insensitive
```

```
$text = "I like Apple, apple juice, and APPLE pie.";
```

```
// Replace all occurrences of "apple" with "mango"
```

```
$result = preg_replace($pattern, "mango", $text);
```

```
echo $result;
```

```
?>
```

- The pattern `/apple/i` matches “Apple”, “apple”, and “APPLE”.
- `preg_replace()` replaces all matches with “mango”.



Example: preg_split()

The `preg_split()` function splits a string into an **array** using a regular expression pattern as the delimiter.

```
<?php
```

```
$pattern = "/[\s,]+/";    // split by spaces or  
commas
```

```
$text = "apple, banana orange,grape";
```

```
$result = preg_split($pattern, $text);
```

```
print_r($result);
```

```
?>
```



Example: preg_grep()

The `preg_grep()` function searches an **array of strings** and returns only the elements that match a given regular expression.

```
<?php
```

```
$pattern = "/php/i";    // case-insensitive  
search for 'php'
```

```
$subjects = array("I love PHP", "Java is  
great", "Learning php is fun", "Python is  
powerful");
```

```
$result = preg_grep($pattern, $subjects);
```

```
print_r($result);
```

```
?>
```



Example: preg_filter()

The `preg_filter()` function searches an **array or string** for matches to a pattern and **replaces** them with the specified replacement. Unlike `preg_replace()`, it returns only the **modified elements** (ignores those without matches).

```
<?php
```

```
$pattern = array("/php/i", "/java/i");  
$replacement = array("PHP (updated)", "Java  
(updated)");
```

```
$subjects = array("I love PHP", "Java is great",  
"Python is powerful");
```

```
$result = preg_filter($pattern, $replacement,  
$subjects);
```

```
print_r($result);
```



Commonly Used Regular Expression Patterns - Anchors and Wildcards

Pattern	Description	Example Match
<code>.</code>	Any single character (except <code>\n</code>)	<code>a.b</code> matches <code>acb</code> , <code>a1b</code>
<code>^</code>	Start of a string	<code>^Hello</code> matches <code>Hello world</code>
<code>\$</code>	End of a string	<code>world\$</code> matches <code>Hello world</code>



Commonly Used Regular Expression Patterns - Quantifiers

Pattern	Description	Example Match
*	Zero or more of preceding element	a* matches "", a, aaa
+	One or more of preceding element	a+ matches a, aa, aaa
?	Zero or one of preceding element	a? matches "", a
{n}	Exactly n occurrences	a{3} matches aaa
{n, }	At least n occurrences	a{2, } matches aa, aaa
{n, m}	Between n and m occurrences	a{2, 4} matches aa, aaa



Commonly Used Regular Expression Patterns - Character Classes

Pattern	Description	Example Match
[]	Any one character from the set	[abc] matches a, b, c
[^]	Any character not in the set	[^abc] matches d, e
\\d	Any digit (0–9)	\\d+ matches 123
\\D	Any non-digit	\\D+ matches abc
\\w	Any word char (letters, digits, _)	\\w+ matches hello123
\\W	Any non-word character	\\W matches @, #
\\s	Any whitespace	\\s+ matches spaces, tabs
\\S	Any non-whitespace	\\S+ matches hello



Commonly Used Regular Expression Patterns - Groups, Alternation, and Flags

Pattern	Description	Example Match
	Alternation (OR)	cat dog matches cat or dog
()	Grouping	(ab) + matches ab, abab
(?i)	Case-insensitive flag	(?i) php matches php, PHP



Task 1: Replace digits with

Write a PHP program that replaces **all digits** in a string with #.

Input:*

My phone number is 9876543210.

Expected Output:

My phone number is #####.



Task 2: Censor bad words

Write a PHP program that replaces the words **bad**, **ugly**, and **stupid** with ***.

Input:

That was a bad idea, really stupid and ugly!

Expected Output:

That was a *** idea, really *** and ***!



Task 3: Format dates

Write a PHP program that converts dates from the format **DD/MM/YYYY** to **YYYY-MM-DD**.

Input:

Today's date is 07/09/2025.

Expected Output:

Today's date is 2025-09-07.

